Cursor 実践コース　第 1 回 / 全 5 回　・　90分

Cursor の基本操作



壊れた買い物カートのページを、AI に直させます。コードを直すと、画面が直る。そこまでを 90 分で通します。

イントロ

このコースについて

今日

第 1 回

基本操作

AI に不具合を直させて、その差分を読める。

第 2 回

バイブコーディング

何も決めずに頼むと何が起きるかを説明できる。

第 3 回

仕様駆動開発

要件を書いてから作らせられる。

第 4 回

チーム開発（前半）

チームで仕様を書き、PR を出せる。

第 5 回

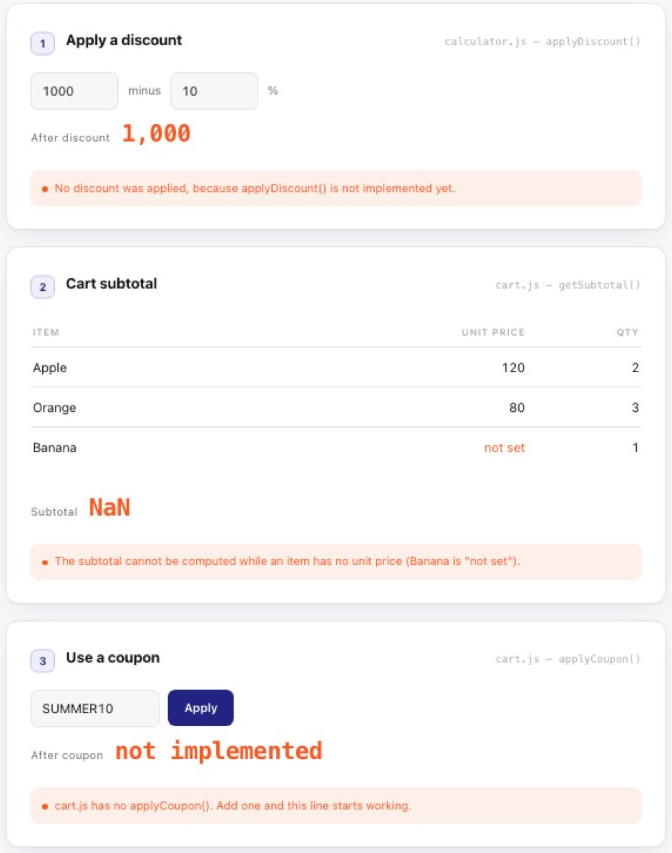
仕上げと発表

動くものをデモできる。

5 回で「チームでアプリを作って発表できる」ところまで行きます。

イントロ

今日やること



壊れた買い物カートのページを、AI に直させます。practice/index.html を開くと、オレンジで 3 か所が壊れています。**画面は英語です。**

① Apply a discount（割引を計算する）

1,000 の 10% 引きなのに **After discount 1,000** のまま

② Cart subtotal（カートの小計）

Subtotal NaN

③ Use a coupon（クーポンを使う）

not implemented

この 3 つが紺色の金額に変わったら、今日は達成です。

イントロ

今日の進め方

01

全部覚えなくていい

今日使う道具は3つだけです。

02

見ているだけの時間はほぼありません

90 分のうち 75 分は自分の手を動かします。

03

詰まったら隣の画面を見る

それでも分からなければ手を挙げてください。

今日の進み方

90分の流れ

時刻	章	内容	主体
0:00	（イントロ）	表紙・コース全体・今日やること（ダウンロード待ちに）	講師
0:00	第1章　環境をそろえる	インストール・クローン・起動（15分）	全員
0:15	第2章　今日のゴール	今日の4段（5分）	講師
0:20	第3章　三つの道具	Tab / Ctrl+K / Agent を1つずつ触る（10分）	全員
0:30	**第4章　Ask と Agent**	**Ask で聞く → Agent で実装する（15分）**	全員
0:45	**第5章　差分を読む**	**自分の diff を読んで Keep する（10分）**	全員
0:55	第6章　ハンズオン	practice/ 課題を自分でやる（25分）	全員
1:20	第7章　持ち帰るもの	今日おぼえて帰ること・次回予告（10分）	講師

> **受講者が手を動かすのは 75 分 / 90 分**です。**イントロは時間枠を持ちません**——第1章のダウンロード待ちに重ねて話します。

Cursor 実践コース　第 1 回 — 基本操作

第 1 章 ／ 0:00 – 0:15

環境をそろえる

Cursor を入れて、教材のページを開くところまで。全員そろってから先へ進みます。

0:00 — セットアップ

Cursor を入れて、教材を落とします

1 cursor.com からダウンロードしてインストールする

OS に合わせて選ぶ。Mac は Apple Silicon 版と Intel 版がある

2 GitHub か Google のアカウントでサインインする

右上にアカウントアイコンが出れば完了

3 教材を落とす（Cursor の中だけでできる）

Ctrl + Shift + P（Mac は Cmd + Shift + P）→ **Git: Clone** → 下の URL を貼る → 置き場所を選ぶ。ターミナル派は git clone でも可

```
https://github.com/xrnd-tec/cursor-course.git
```

Node.js は当日入れなくて構いません。教材のページはブラウザだけで動きます。挑戦枠の課題 C でだけ node を使いますが、無ければ飛ばして問題ありません。

0:00 — セットアップ

教材のページを内蔵ブラウザで開きます

1 **Cursor で cursor-course/ フォルダを開く**

File → Open Folder から選ぶ

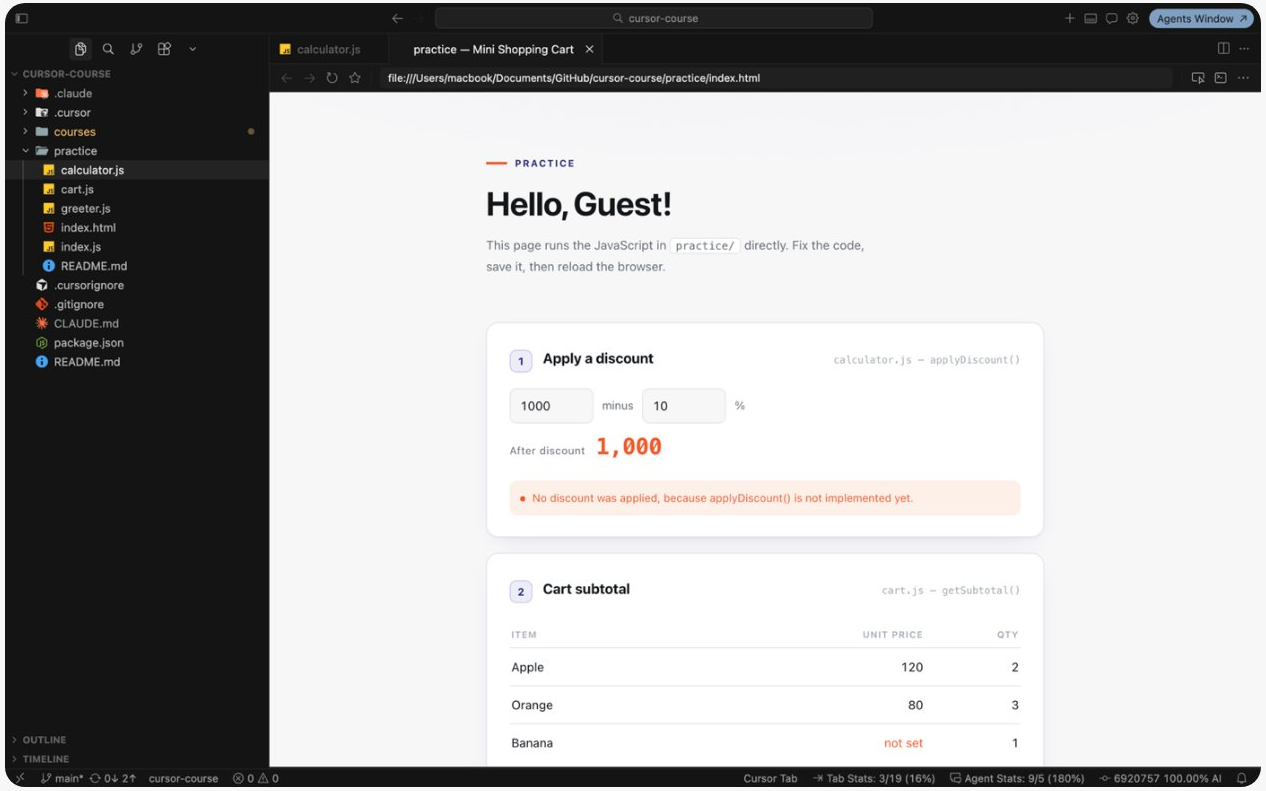
2 **右クリック → Open In Browser**

サイドバーで practice/index.html を右クリック。Cursor の内蔵ブラウザで開く

3 **「ミニ買い物カート」が出たら準備完了**

①②③ がオレンジで表示されている状態

OS のブラウザで直接開くと、数字が「…」のまま出ません。ES モジュールがブロックされるためです。必ず Cursor の内蔵ブラウザで開いてください。



0:00 — 最初にそろえる（必須）

起動を IDE ビューにします

ページが開けたら、**今日使う画面だけ全員で揃えます。**

1. **Cursor Settings → General → Startup → Window Restoration**

   Last Used Windows にする

2. **IDE ビュー（中央にコード）であることを確認する**

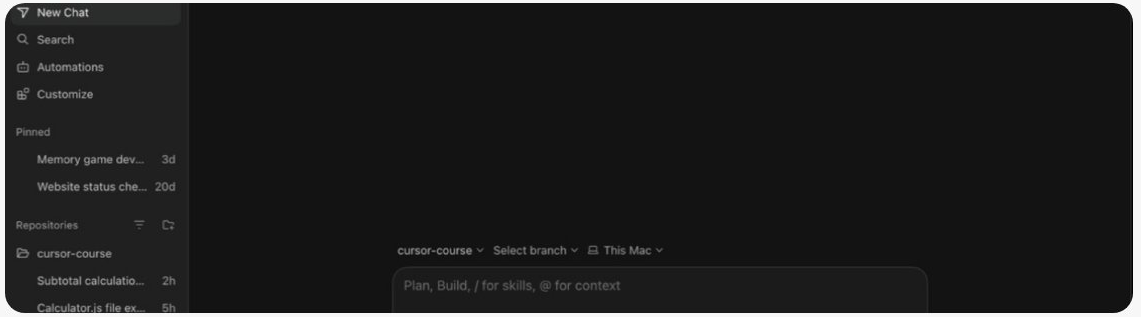
   Ctrl+Shift+P（Mac は Cmd+Shift+P）→ Open IDE

3. **次回も IDE で開くように、IDE から Cursor を終了する癖をつける**

   今日は終了しなくてよい

起動したときに見慣れない画面（Agents Window）が出ることがあります。**壊れていません。**今日使うのは IDE（中央にコード）です。

起動直後（Agents Window）



IDE ビュー（中央にコード）



0:00 — コラム（任意）

先に終わった人へ

ここから 2 つは**任意**です。今日の実習には要りません。先に終わった人がやるか、持ち帰りで構いません。

コラム 1

表示言語を日本語にする

IDE で **Ctrl + Shift + P**（Mac は Cmd + Shift + P）→ `Configure Display Language` → **Japanese** → 完全再起動。

VS Code 由来のメニューは日本語になりますが、**Agent / Cursor Settings は 2026 年 8 月時点ではほぼ英語のまま**です（設定ミスではありません）。

コラム 2

Auto-review（個人開発向け）

Settings → **Agents** → **Execution and Approvals** の Run Mode を **Auto-review** にすると、安全そうなコマンドは自動、危険度が高ければ確認、になります。

今日の実習では必須ではありません。

0:00 — セットアップ確認

ここまで揃えてください

- [ ] Cursor が起動している
- [ ] サインインが完了している（右上にアカウントアイコン）
- [ ] **IDE ビュー**で cursor-course/ を開いている
- [ ] 内蔵ブラウザに「ミニ買い物カート」が出ている
- [ ] ページの①②③がオレンジで表示されている

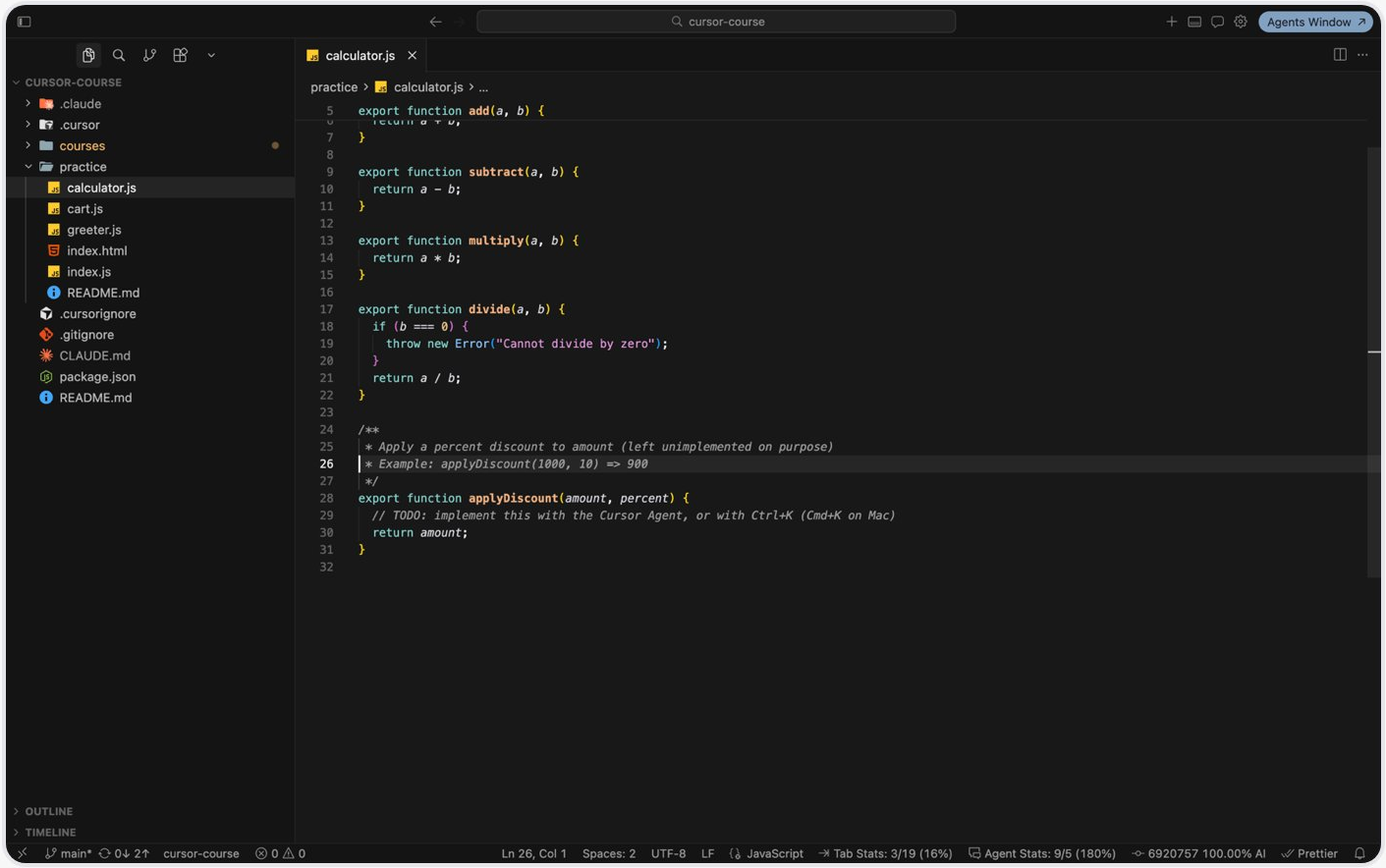
詰まったら

ダウンロードが遅い	全員同時だと詰まる。待つ
サインインできない	GitHub / Google で作成
見慣れない画面が出た	Ctrl+Shift+P で IDE に戻す
数字が「…」のまま	内蔵ブラウザで開き直す
メニューが英語のまま	コラム 1 の手順で日本語にできる

全員がここに到達するまで、第2章へ進みません。先に終わった人は止めなくて構いません。

0:00 — セットアップ確認

今日はこの画面で作業します



サイドバー

practice/ の中のファイルを開く。消えたら Ctrl+B（Mac は Cmd+B）。

エディタ

今日さわるのは calculator.js・greeter.js・cart.js の3つだけ。

Agent は、まだ閉じたまま

右の Agent パネル。Ctrl+I（Mac は Cmd+I）で開く。第3章から使います。

Cursor 実践コース　第 1 回 — 基本操作

第 2 章 ／ 0:15 – 0:20

今日のゴール

4段のはしご。どこまで登るかを先に決めます。この章は聞くだけです。

0:15 — 今日のゴール

今日できるようになること

01 **Cursor の基本操作を覚える**
第3章 — 見出しの挨拶が変わる

02 **既存のコードを AI に説明させる**
第4章（Ask で @ を付けて聞く）

03 **不具合を AI に直させる**
第6章・課題 A — ② Subtotal が出る

04 **新しい機能を AI に実装させる** 挑戦
第4章 / 第6章・課題 B — ① After discount が変わる / ③ クーポンが動く

第4章で applyDiscount を実装するので、04 の入口は第4章で全員が一度通ります。03 が全員の到達ラインです。

Cursor 実践コース　第 1 回 — 基本操作

第 3 章 ／ 0:20 – 0:30

三つの道具

Tab、Ctrl+K、Agent。3つを1つずつ、自分で触ります。

0:20 — 第3章

AI に頼む手段は3つです

Tab

続きを提案

入力中に、灰色で続きが出る。Tab で採用、Esc で却下。周囲のコード・最近の編集を見ている。

Ctrl + K

ここだけ直す

選択した範囲だけを直す。「この関数を async に」「この文言を変えて」。Mac は Cmd + K。

Ctrl + I

会話しながら変える

右の Agent パネル。複数ファイルを読んで、答えて、直す。今日はこれが主役。Mac は Cmd + I。

> **このコースのショートカット表記は Windows が基本です。**Mac の人は括弧内を読んでください。
> 今日の画面は **IDE ビュー（中央にコード）＋ 右の Agent パネル。**Agents Window は今日は開きません。

0:20 — 第3章

押すキーの早見表

やりたいこと	画面からやる	Windows	Mac
サイドバーを出す / 隠す	左上のアイコン	**Ctrl + B**	Cmd + B
Agent の入力欄を開く	右の Agent パネル	**Ctrl + I**	Cmd + I
モードを変える	入力欄左下のモード名をクリック	**Shift + Tab（順送り）**	Shift + Tab
モデルを変える	入力欄のモデル名をクリック	**Ctrl + /**	Cmd + /
選んだ範囲だけ直す	（ショートカットのみ）	**Ctrl + K**	Cmd + K
続きを受け取る	（ショートカットのみ）	**Tab**	Tab
IDE ビューに戻す	コマンドパレット → Open IDE	**Ctrl + Shift + P**	Cmd + Shift + P

> ショートカットは自分で変更できます。**効かないときは画面のボタンから操作して構いません。**ここで止まると後が続きません。

0:20 — 第3章

Tab が得意なこと、任せないこと

Tab が見ているもの：周囲のコード・最近の編集・リンターの情報。だから「いま書いている流れの続き」に強い。

Tab が活きるとき	Tab に任せないとき
似たパターンの繰り返し	仕様判断が必要なロジック
import や型の補完	セキュリティやお金の計算
「今書いてる流れ」の続き	レビューが必須の箇所

> もっと詳しく：courses/fundamentals/00-map.md ・ 04-tab-and-inline.md

0:20 — 第3章 ①（3分）

まず Tab を出してみます

practice/calculator.js の applyDiscount の中にカーソルを置いて、何か書きかけてください。

```
  return a / b;
}

/**
 * Apply a percent discount to amount (left unimplemented on purpose)
 * Example: applyDiscount(1000, 10) => 900
 */
export function applyDiscount(amount, percent) {
export function    applyDiscount (amount, percent) {
  const discounted =  amount * (1 - percent / 100);
  return discounted;
}
```

> 灰色は「まだ入っていない文字」。**Tab** で採用、**Esc** で却下。受け取るまでファイルは変わりません。

0:20 — 第3章 ②（5分）

Ctrl + K で 1 か所だけ直します

practice/greeter.js を開き、greet 関数全体を選択して **Ctrl + K**（Mac は Cmd + K）。下を貼り付けて送ります。

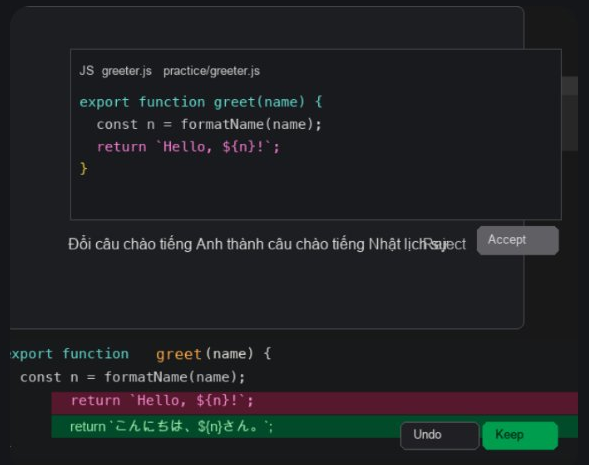
英語の挨拶を、丁寧な日本語の挨拶に変えて。

> 出てきたら **Accept**（採用）か **Reject**（取り消し）。ここだけは Keep / Undo ではなく Accept / Reject 表記です。

① 選んで Ctrl + K



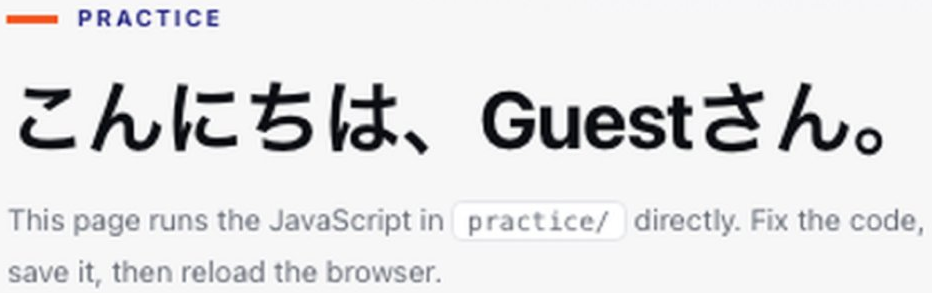
② その場で差分になる



0:20 — 第3章 ②

Accept したら、画面を再読み込みします

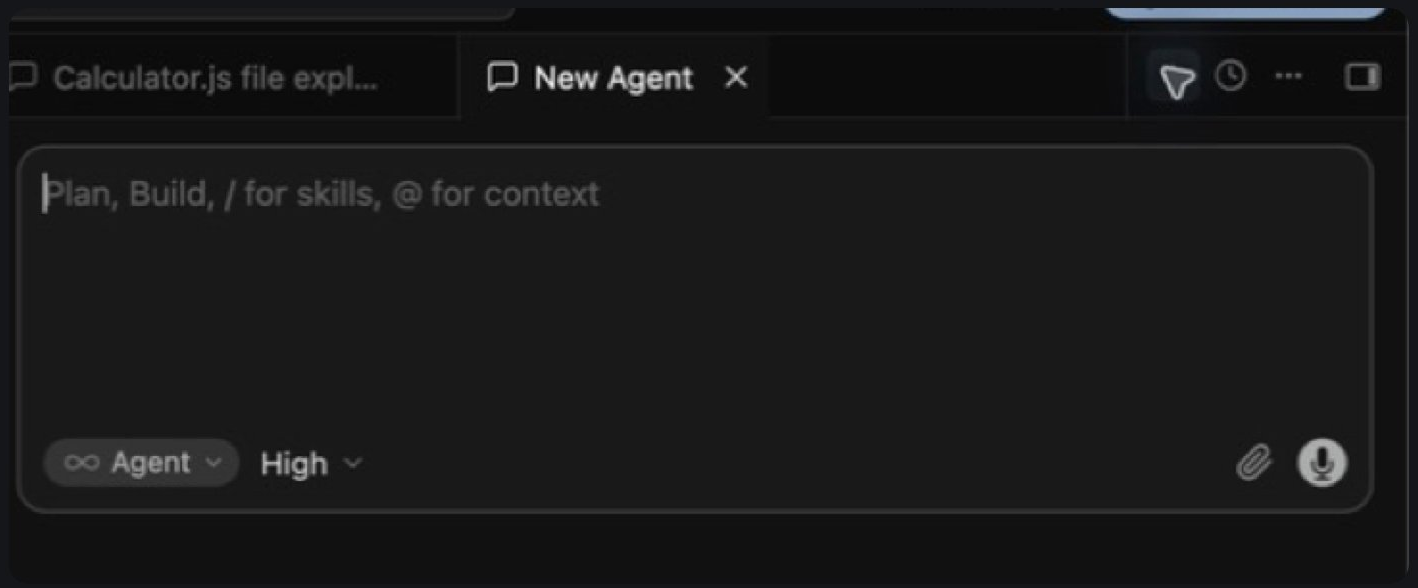
ブラウザを再読み込みすると、**ページの見出しが変わります。**



今日はじめて**自分の変更が画面に出る**瞬間です。「いま直したのはコードですが、変わったのは画面です。」

0:20 — 第3章 ③（2分）

Agent の入力欄を開くだけ



> **Ctrl + I**（Mac は Cmd + I）で右の Agent パネルが開きます。Agents Window は今日は使いません。**まだ何も送りません。**開くだけです。

0:20 — 第3章

使い分けの早見

今打っている続きが欲しい	**Tab**
この選択範囲だけ直したい	**Ctrl + K**
調査したい / まだ書くな	**Ask**
作って・直して・動かして	**Agent**
大きいので方針から	**Plan**

> Tab は「続き」、Ctrl + K は「ここだけ」、Agent は「会話しながら」。**今日は主に Agent を使います。**

Cursor 実践コース　第 1 回 — 基本操作

第 4 章 ／ 0:30 – 0:45

Ask と Agent

今日の中心です。Ask で調べてから Agent で直す。この1サイクルを自分の手で回します。

0:30 — 第4章

モードは AI の振る舞いを決めます

モード	向いていること	ファイル編集
Agent	機能追加・リファクタ・バグ修正・テスト実行	する
Plan	大きい変更の方針を先に合意してから実装	計画承認後に実装
Debug	再現が難しいバグを、証拠を取りながら追う	する
Multitask	独立した複数の作業を並列で進める	する（並列に）
Ask	調査・説明・設計の相談（まだ書かせたくない）	**しない（読み取りのみ）**

> 入力欄の左下に出ていて、クリックで選ぶか **Shift + Tab** で順送りできます。**今日は Ask と Agent の2つだけ**使います。

0:30 — 第4章

モードとモデルは別物です

モード

AI の振る舞い

編集するか、計画を先に出すか。
入力欄の左下。**Shift + Tab** で順送り。

モデル

依頼を処理する頭脳

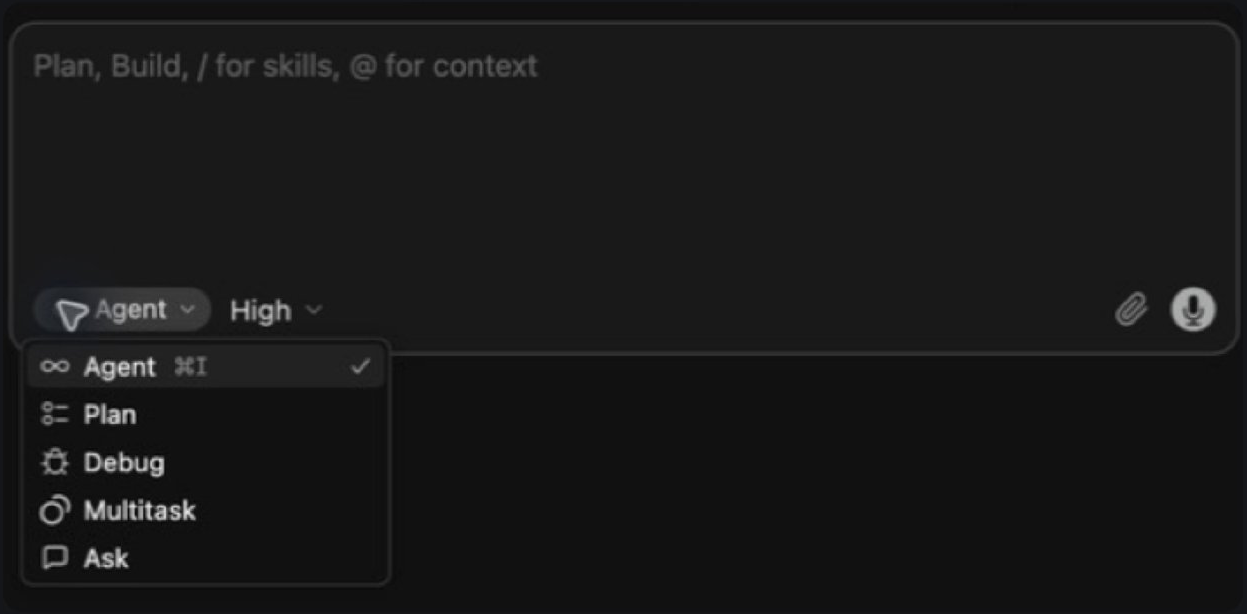
どの AI が答えるか。
入力欄の近く。**Ctrl + /**（Mac は Cmd + /）。

> 既定の **Auto** は「モデルを選ばない」ではなく、**Cursor Router がリクエストごとに自動で選んでいる**という意味です。難しいタスクは高性能なモデルへ、簡単なものは低コストなモデルへ振り分けられます。

今日はそのままで構いません。

0:30 — 第4章 ①（2分）

モード名をクリックして Ask を選びます



入力欄**左下**のモード名をクリックすると、この一覧が開きます。

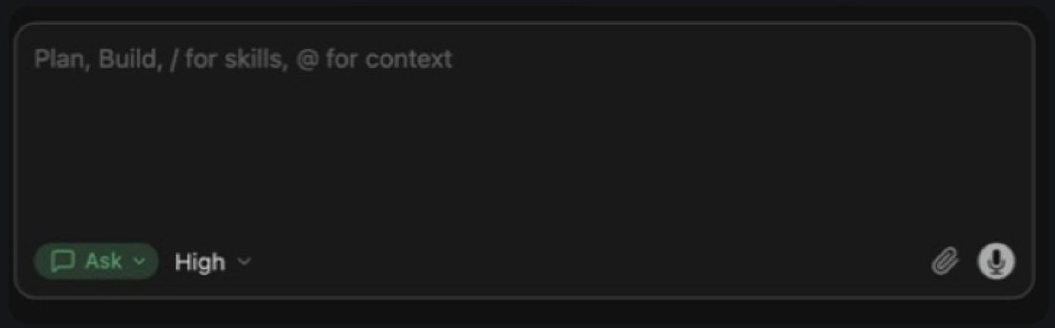
今日使うのは **Ask** と **Agent** の2つだけです。

Shift + Tab でも変わりますが、一覧は開かず順に切り替わります。

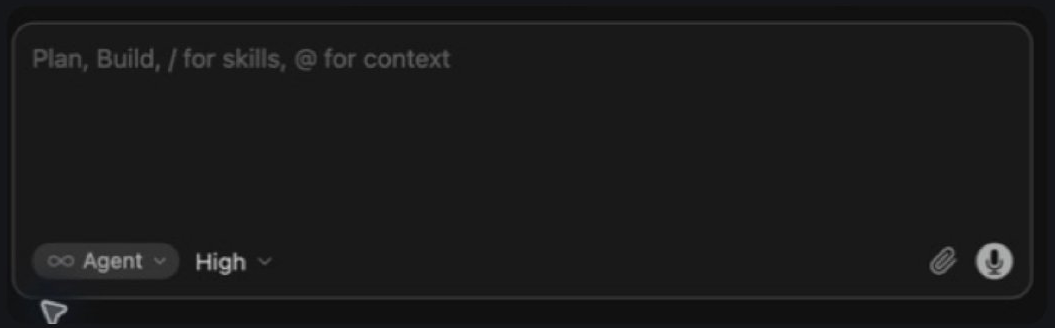
0:30 — 第4章 ①

切り替わったかは、ここを見ます

Ask になっている



Agent に戻っている



> 入力欄の左下が今のモードです。**送る前に必ず自分の目で確認してください。**Ask のまま実装を頼むのが最初の詰まりどころです。

0:30 — 第4章

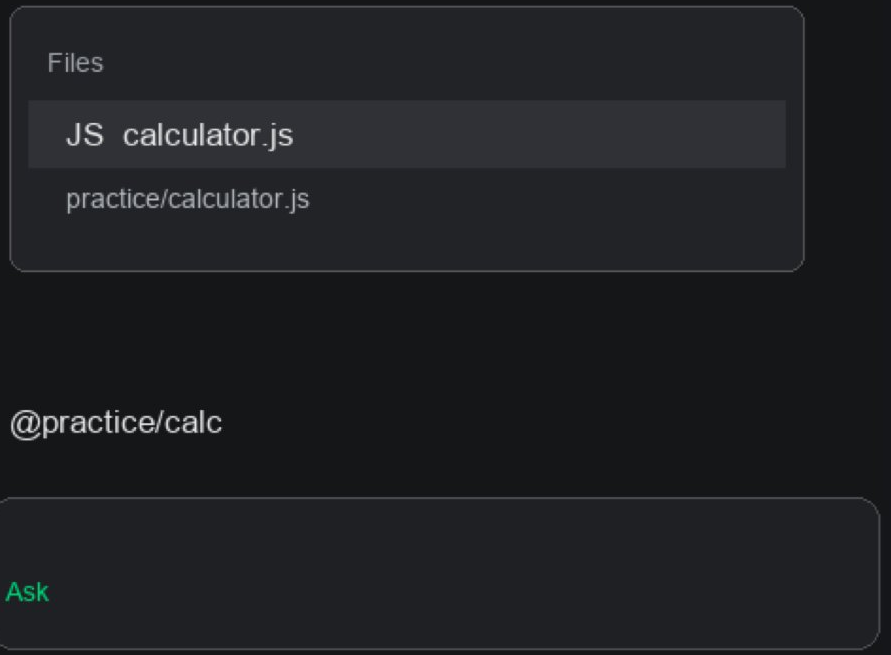
@ で、会話に載せる情報を指定します

指定	使いどころ
@ファイル名	「このファイルを前提に話して / 直して」
@フォルダ/	「この範囲だけ見て」
@Docs	登録済みドキュメントを参照
@Terminals	エラー出力や実行結果を見せる
@Past Chats	以前の会話の続き
@Commit / @Branch	未コミット差分や main との差分

> 考え方は「**関係ファイルが分かっているなら @ する。分からないなら付けず、Agent に探させる**」。無関係な大きいフォルダを全部 @ するとノイズになります。**今日は1ファイルだけで十分です。**

0:30 — 第4章

@ を打つと候補が出ます



> @ を打つと候補が出ます。ここで選ばないと、Cursor はファイルを読まずに答えます。

0:30 — 第4章

同じ依頼でも、書き方で結果が変わります

弱い依頼

バグ直して

良い依頼

```
@practice/cart.js @Terminals
「小計が NaN になる」問題を直して。
再現手順: addItem に price 未定義の商品を
入れたあと getSubtotal を呼ぶ。
```

> 対象・症状・再現手順。この3つがあるだけで返ってくるものが変わります。

0:30 — 第4章 ②（4分）

Ask で聞きます

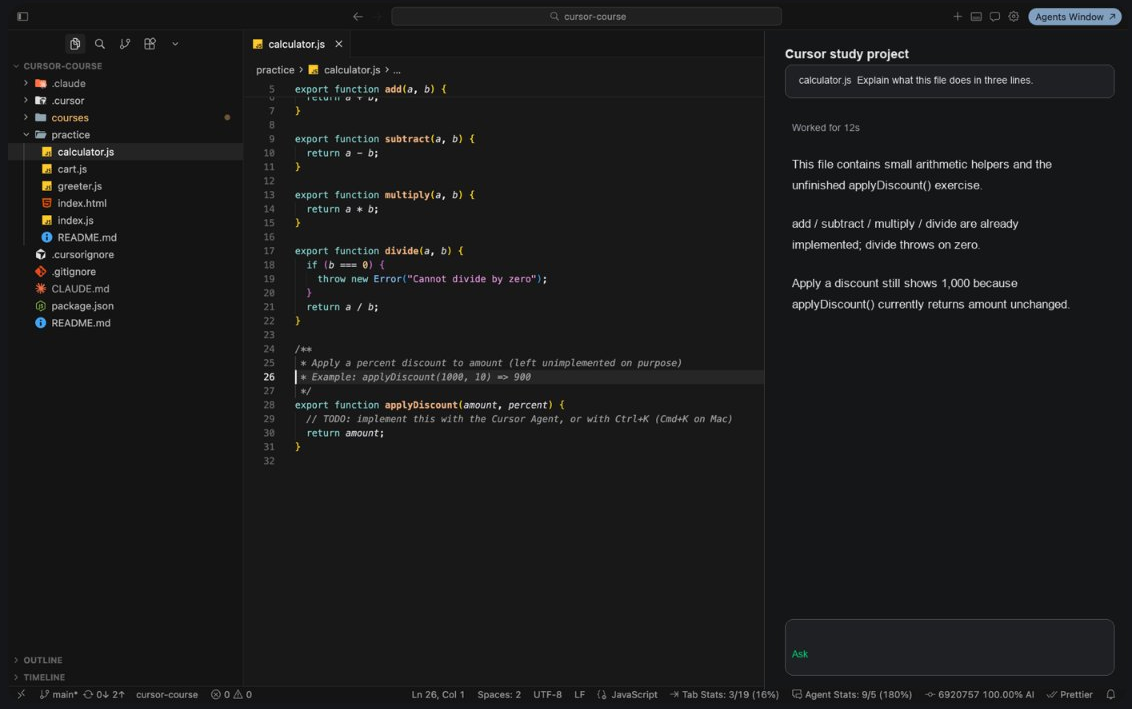
1. Ctrl + I（Mac は Cmd + I）で右の Agent パネルを開く
2. 左下のモード名をクリックして **Ask** を選ぶ
3. 下のプロンプトを送る

```
@practice/calculator.js
このファイルの役割を3行で説明して。
そのうえで、画面の Apply a discount が効かない理由を教えて。
```

> 送ったら 30〜60 秒返ってきません。@ で渡せるのはファイルだけでなく、フォルダやエラー出力もです。ただし今日は1ファイルで十分。

0:30 — 第4章 ②

Ask は答えるだけです



回答は返ってきますが、**エディタ側は何も変わっていません。**

差分も出ていません。

中身を知りたいときは Ask、変えてほしいときは Agent です。

0:30 — 第4章 ③④（9分）

Agent に戻して、実装させます

モード名をクリックして **Agent** に戻してから送ります。

```
@practice/calculator.js
applyDiscount(amount, percent) を実装して。
例: applyDiscount(1000, 10) → 900
TODOコメントは残さなくてよい
```

対象を渡す
@ でファイルを明示する

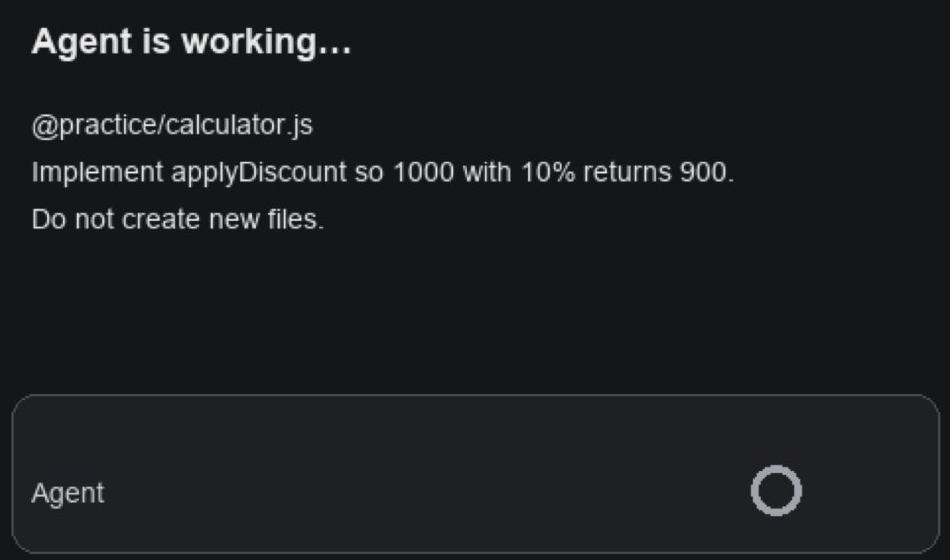
例を書く
入力と期待する出力を1つ

完了条件を書く
どうなったら終わりかを1行

差分が出るまで待ちます。まだ Keep しないでください。次の章で読んでから決めます。

0:30 — 第4章 ④

送ったら、待ちます



考えている間はこう見えます。

30〜60 秒かかることもあります。

止めたいときは入力欄の右の停止ボタンを押してください。

Cursor 実践コース　第 1 回 — 基本操作

第 5 章 ／ 0:45 – 0:55

差分を読む

自分が出した差分を読んで Keep します。今日いちばん大事な章です。

0:45 — 第5章

変更は、もうファイルに入っています

「提案を見てから反映」ではありません。Agent の変更は作業中にファイルへ適用されていきます。だから、読んでから決めます。

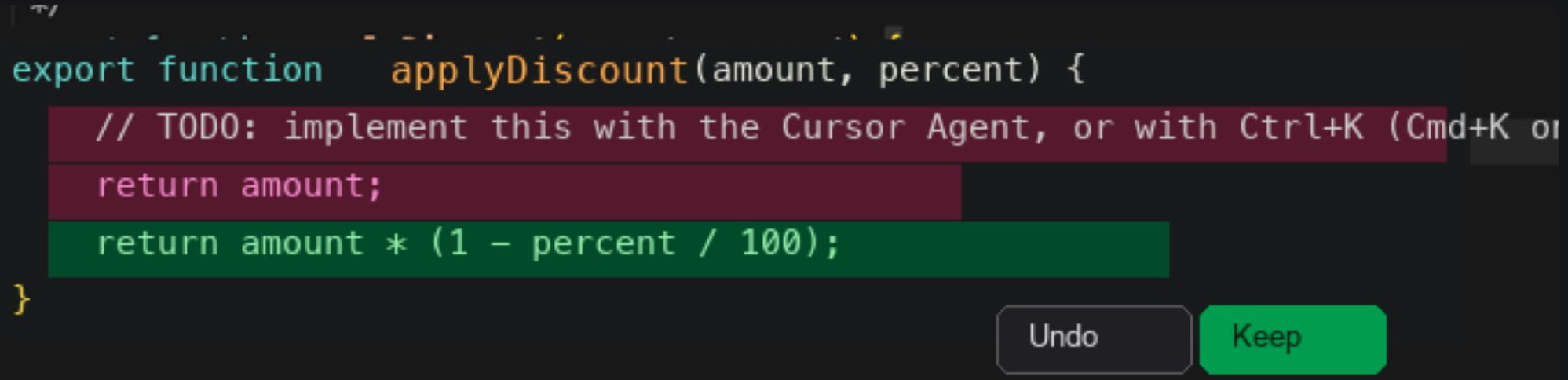
決めるボタン	意味	どこにある
Keep	採用する	差分の右下、または Agent パネル下部の変更バー
Undo	取り消して元に戻す	同上
Review	変更全体をまとめて確認する	Agent パネル下部
Restore Checkpoint	その時点まで大きく巻き戻す	対象メッセージの横

> **緑**が追加された行、**赤**が削除された行。「全部受け入れたつもり」でも、git や目視で最終確認する癖をつけてください。

0:45 — 第5章 ①（4分）

これが diff です

赤が消える行、緑が入る行。**自分が出した差分**を1行ずつ読みます。



> 下の **Undo / Keep** で、この差分を採るか戻すかを決めます。**押す前に緑と赤を読みます。**

0:45 — 第5章 ①

読むときに自分へ問う3つ

01

頼んだこと以外が変わっていないか

余計なファイル・余計な行が混ざっていないか。

02

消えている行（赤）は、消えて困らないか

赤は元に戻せるうちに読む。

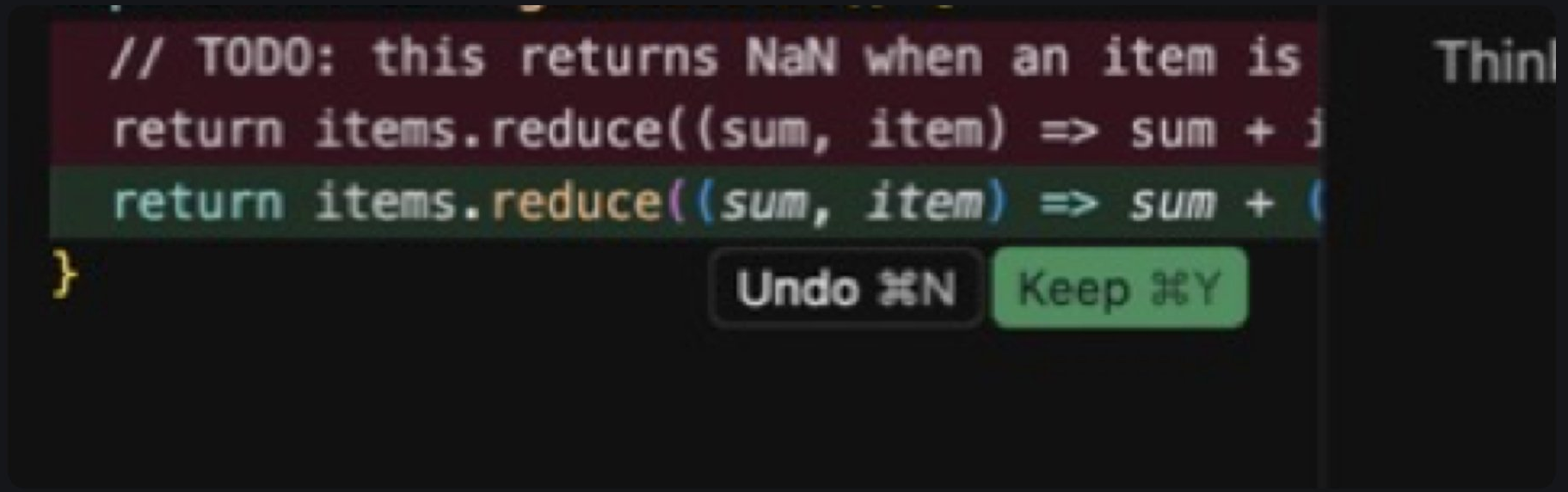
03

applyDiscount(1000, 10) が 900 になりそうか

コードを読んで、期待値になるかを自分で判断する。

0:45 — 第5章 ②（2分）

Keep か Undo かを決めます



> 納得できたら **Keep**、できなければ **Undo** で戻してもう一度頼み直します。大きく戻したいときはメッセージ横の **Restore Checkpoint**。

0:45 — 第5章 ②

Keep するとファイルが変わります

```
export function applyDiscount(amount, percent) {
  return amount * (1 - percent / 100);
}
```

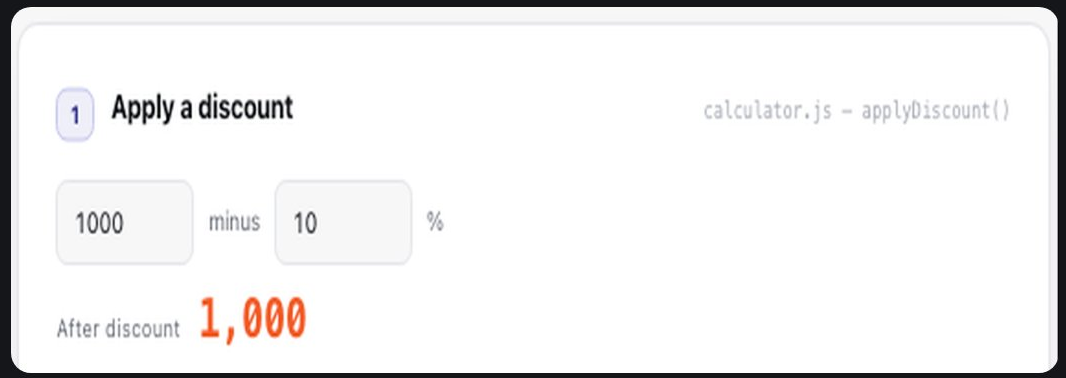
> 色が消えて、applyDiscount が実装済みになりました。**ただし、まだ画面は見ていません。**

0:45 — 第5章 ③（2分）

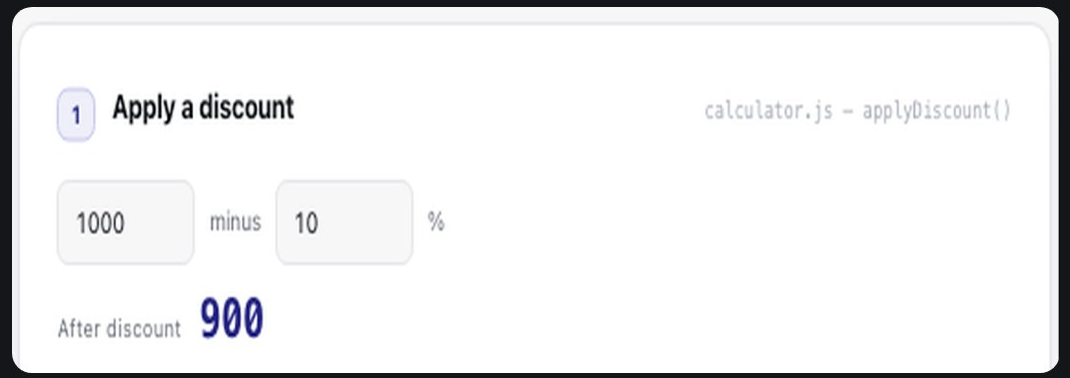
画面で確かめます

ブラウザを再読み込みしてください。

直前まで



再読み込み後



コードを直したら、画面が直りました。ここまでが1サイクルです。数字が変わらなければ、Keep できていないか、実装が違っています。

0:45 — 第5章 ④（2分）

Ask で答え合わせをします

モードを **Ask** に切り替えて送ります。

```
@practice/calculator.js
applyDiscount(1000, 10) の期待値は何？ 今の実装で合ってる？
```

> 直したあとに、もう一度 Ask で確かめる。これも1サイクルの一部です。

今日の型

必ずこの順で回します

01
Ask
@ファイル ＋ 質問

先に中身を知る

02
Agent
@ファイル ＋ 依頼 ＋ 完了条件

変えてもらう

03
diff
緑と赤を1行ずつ読む

頼んだ通りか確かめる

04
Keep / Undo
残すか、戻すか

自分で決める

> **Keep する前に必ず diff を読む。**これが Cursor を安全に使う土台になります。

Cursor 実践コース　第 1 回 — 基本操作

第 6 章 ／ 0:55 – 1:20

ハンズオン

ここからは講師は話しません。25分、全員が自分で手を動かします。

0:55 — 第6章

頼み方には型があります

モデルより先に「ゴール・制約・完了条件」を書く。今日の課題のプロンプトも、この形になっています。

```
【やりたいこと】一言
【対象】@ファイル or フォルダ（分かる範囲）
【制約】壊したくないもの / 使ってよい技術
【完了条件】何があれば終わりか
【やらなくてよいこと】（任意）
```

避ける言い方	代わりに
いい感じにして	具体的な見た目・挙動・境界条件
全部リファクタ	対象ファイルと「やる / やらない」
バグ直して	期待・実際・再現手順
最適に	速度 / 可読性 / 互換のどれ優先か

> **タスクが変わったら新規チャット。**古い前提が邪魔になりにくくなります。

0:55 — 課題 A（全員・最低ライン）

cart.js の小計を直してください

ゴール 03 — 画面②。Subtotal が **NaN** になっています。第4章と同じ順番で、Ask で確認してから Agent に頼みます。

① Ask

```
@practice/cart.js
getSubtotal の現状の問題点を1〜2個教えて。
直す前に確認したいだけ。
```

差分を読んで **Keep** したら、**ブラウザを再読み込み**してください。

② Agent

```
@practice/cart.js
getSubtotal を次の条件で改善して。
- price か qty が undefined / null の
  アイテムはスキップ
- 空カートが 0 を返す今の挙動は変えない
- 既存の関数名・引数は変えない
```

直前まで

② Subtotal **NaN**（オレンジ）

再読み込み後

② Subtotal **480**（紺）

0:55 — 課題 A

こうなれば正解です

```
export function getSubtotal() {
  // TODO: this returns NaN when an item is missing price or qty. Make it survive that
  return items.reduce((sum, item) => sum + item.price * item.qty, 0);
  return items.reduce((sum, item) => {
    if (item.price == null || item.qty == null) {
      return sum;
    }
    return sum + item.price * item.qty;
  }, 0);
}
```

Undo Keep

> price か qty が欠けたアイテムを飛ばす処理が入っていれば大丈夫です。書き方は違っていて構いません。

0:55 — 課題 B（余裕があれば）

クーポンを実装します

ゴール 04 — 画面③。挑戦枠です。課題 A が終わってから取りかかってください。

```
@practice/cart.js @practice/calculator.js
cart.js に applyCoupon(code) を追加して。
- SUMMER10 のとき getSubtotal の 10% 引きを返す
- 無効なコードのとき getSubtotal をそのまま返す
- calculator.js の applyDiscount を使ってよい
- 新規ファイルは作らない
```

Keep して再読み込みすると、画面③の **not implemented** が消えて金額が出ます。**これで 3 か所すべてが紺色になります。**

※ applyDiscount が未実装だと動きません。第4章で Undo した人は、先に実装してください。関数名は **applyCoupon**、**export** も要ります。

0:55 — 課題 C（さらに余裕があれば）

エラーから直す型をおぼえます

同じロジックを、今度は**ターミナルで**動かします。**practice/index.js** はまだ空のプレースホルダです。

```
@practice/index.js @practice/cart.js
index.js に、カートの中身と小計を console.log で出す処理を書いて。
完了条件: node practice/index.js がエラーなく動き、金額がログに出る
```

ターミナルで動かす

```
node practice/index.js
```

エラーが出たら、ターミナルの出力をそのまま Agent に貼り付けて直してもらってください。これがエラー対応の型です。

※ node が入っていない人は飛ばして構いません。

Cursor 実践コース　第 1 回 — 基本操作

第 7 章 ／ 1:20 – 1:30

持ち帰るもの

今日おぼえて帰ること、提出、そして次回。

1:20 — 第7章

今日おぼえて帰ること

01

Ask で確認してから Agent で依頼する

いきなり変えない。中身を知ってから頼む。

02

@ で対象を明示する

曖昧な依頼は、曖昧な結果になる。

03

diff を読んでから Keep

Cursor を信頼しすぎない。決めるのは自分。

1:20 — 提出（30秒）

画面のスクショを1枚ください

カートのページのスクリーンショットを1枚、
チャットか共有フォルダへ。

オレンジが1つでも紺に変わっていれば十分です。全部直っていなくて大丈夫。

何を見るか

オレンジが何個残っているかで、誰がどこまで直せたかが一目で分かります。**第2回の入り方を決める材料**にします。

1:20 — 第7章

既定でどこまで守られているか

今日は学習用のリポジトリなので何をしても大丈夫です。ただし業務のリポジトリでは、API キーや顧客データを Agent に読ませないよう気をつけてください。

対象	既定の扱い
ファイルの読み取り・検索	承認不要
ファイルの書き換え	即座に保存される。だからバージョン管理が前提
ターミナルのコマンド実行	**Run Mode に従う**（個人開発なら Auto-review）
ネットワーク	任意の通信は不可。GitHub・直接指定したリンク・Web 検索などに限られる

> 読ませたくないものは **.cursorignore** で外せます。ただし**秘密はそもそもリポジトリに置かない**のが最優先です。個人開発なら Auto-review（第1章のコラム 2）がおすすめです。

次回予告 — 第 2 回

次回は神経衰弱を作ります

何も決めずに「いい感じに作って」と頼んで作り、
最後に全員のゲームを見せ合います。

復習（自習用）

詰まったところだけ調べてください

00-map.md	全体像・最初の設定	Tab / Ctrl+K / Agent の使い分け、起動 / 日本語 / Auto-review
01-modes.md	モード詳細	Agent / Ask / Plan / Debug と Multitask
03-context.md	@ の全種類	ファイル・フォルダ・エラー出力の渡し方
05-prompting.md	うまい頼み方	曖昧な依頼を、そのまま実行できる形に変える

courses/fundamentals/ 以下にあります